UFR d'Ingénierie

10 décembre 2025

L2 Informatique (2025-2026)

Logique (UL2IN007)

TP

Thème 11

Thème 11 - TP

Objectifs

- prolog, listes, récursion

Exercices

Exercice 16 – Tests d'unification

Tester les unifications suivantes : calculer sur papier le résultat que prolog renvoie pour les requêtes suivantes, selon les principes d'unification et de construction de liste. Vérifier ensuite, en utilisant prolog, que vos réponses sont exactes, en examinant toutes les réponses successives lorsque plusieurs peuvent être obtenues.

1. `?- [a, b, c] = [X, Y].`
2. `?- [a, [b, c], d] = [X | Y].`
3. `?- [a, [b, c], d] = [X, [Y], Z].`
4. `?- [a, [b, c], d] = [X, [Y, Z], U].`
5. `?- [a, [b, c], d] = [X, [Y| Z], U].`
6. `?- [a, b, [c, d]] = [a, b | X].`
7. `?- [a, [b, c], d] = [a, b | X].`
8. `?- [a, b, c, d| X ] = [a, b | X].`
9. `?- append([a, X], [c, Y], [a, b, c, d]).`
10. `?- append([X], Y, [a, b, c, d]).`
11. `?- append(X, [Y], [a, b, c, d]).`
12. `?- append(X, Y, [a, b, c, d]).`

NB désormais, il est recommandé de nommer les variables qui doivent s'unifier avec des listes `L` ou `L1`, ... et de garder `X`, `Y`, ... pour des éléments des listes, afin de rendre les programmes plus lisibles.

Exercice 17 – Manipulation de listes

1. Implémenter le prédicat `premier` vu en TD et tester la requête `premier(L, a)`. Justifier le résultat obtenu.
2. Définir et tester de la même façon les prédicats `deuxieme` et `avantDernier`.
3. Définir le prédicat `inverse` tel que `inverse(L1, L2)` est satisfait si `L2` est la liste contenant les éléments de `L1` dans l'ordre inverse.

   Par exemple `inverse([a, b, c, d], L)` doit conduire à l'unique unification `L = [d, c, b, a]`
   et `inverse([], L)` à `L = []`.

Exercice 18 – Listes de courses et menus

Dans cet exercice, on considère que les constantes correspondent à des aliments et que les listes correspondent à des listes de courses ou à des menus, par exemple
`[orange, avocat, pomme, poulet, gateau, gateau, sardine, saumon, riz]` ou `[avocat, poulet, riz, yaourt]`.

On considère de plus que l'on dispose des prédicats unaires `entree`, `poisson`, `viande`, `accompagnement`, `fruit` et `dessert` qui permettent de définir des catégories d'aliments.

1. Télécharger le fichier `aliment.pl` depuis le moodle. Il contient un ensemble de faits pour les prédicats de catégories d'aliments.
2. Définir le prédicat `platPrincipal` tel que `platPrincipal(X)` est satisfait si `X` est une viande ou un poisson.
3. Ecrire une règle qui exprime que les fruits sont des desserts.
4. Définir le prédicat `menuComplet` tel que `menuComplet(L)` est satisfait si `L` est une liste qui décrit un menu complet, c'est-à-dire contenant, dans cet ordre, une entrée, un plat, un accompagnement et un dessert.
5. Quelle requête permet de trouver tous les menus complets possibles avec les aliments définis dans le fichier `aliment.pl` dont le plat principal est de la viande et le dessert un gâteau ?
6. Définir le prédicat `extraitEntreeMenuComplet` tel que `extraitEntreeMenuComplet(L, E)` est satisfait si `L` est une liste qui décrit un menu complet dont l'entrée est `E`.

   Par exemple, `extraitEntreeMenuComplet([avocat, poulet, riz, yaourt], E).` doit fournir pour unique unification `E = avocat`.
7. Définir de même le prédicat `extraitDessertMenuComplet` tel que `extraitDessertMenuComplet(L, D)` est satisfait si `L` est une liste qui décrit un menu complet dont le dessert est `D`.
8. Définir le prédicat `contient` tel que `contient(L, X)` est satisfait si une liste (de courses) `L` contient l'aliment `X` (permettant de vérifier qu'on n'a rien oublié au supermarché).

   Par exemple, pour
   `ListeCourses = [orange, avocat, pomme, poulet, gateau, gateau, sardine, saumon, riz]`,
   la requête `contient(ListeCourses, gateau).` doit réussir, alors que
   `contient(ListeCourses, yaourt).` doit échouer.
9. Définir le prédicat `extraitDessert` tel que `extraitDessert(L1, L2)` est satisfait si `L2` est la liste de tous les desserts présents dans la liste de courses `L1`.

   Par exemple, pour la liste de courses de la question précédente, `extraitDessert(ListeCourses, Ldessert)` doit conduire à l'unique unification `Ldessert = [orange, pomme, gateau, gateau].`
10. Définir le prédicat `supprimeDessert` tel que `supprimeDessert(L1, L2)` est satisfait si `L2` est la liste obtenue après suppression dans `L1` de tous les desserts qu'elle contient.

    Par exemple, pour la liste de courses de la question précédente, `extraitDessert(ListeCourses, Lres)` doit conduire à l'unique unification `Lres = [avocat, poulet, sardine, saumon, riz].`
11. Définir le prédicat `cureVitamineC` tel que `cureVitamineC(L1, L2)` est satisfait si `L2` est la liste obtenue en remplaçant dans `L1` tous les desserts par des oranges.

    Par exemple, pour la liste de courses de la question précédente, `extraitDessert(ListeCourses, Lres)` doit conduire à l'unique unification `Lres = [orange, avocat, orange, poulet, orange, orange, sardine, saumon, riz].`